

Chapter 13 Abstract Classes and Interfaces

Part 2 (Interfaces)



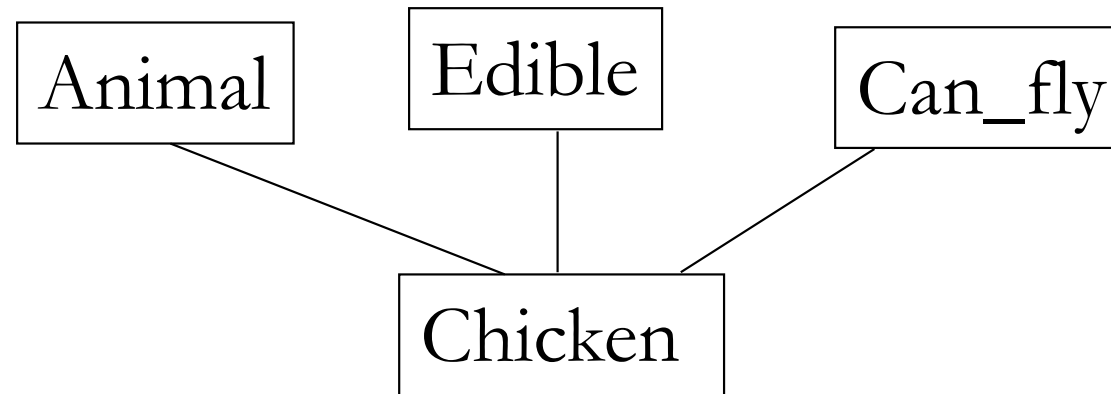
Objectives

- ❑ To specify common behavior for objects using interfaces (§15.5).
- ❑ To **define interfaces** and define classes that implement interfaces (§15.5).
- ❑ To define a natural order using the **Comparable** interface (§15.6).
- ❑ To explore the **similarities and differences** among concrete classes, abstract classes, and interfaces (§15.8).





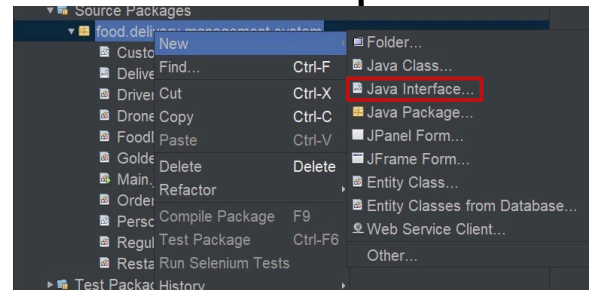
Inheritance



Chicken inherits characteristics from different classes
Animal , Edible , and Can_fly

Java does NOT support multiple inheritances.

But one can use **interface** to implement the functionality of multiple classes.



Interfaces

- What is an interface?
- Why is an interface useful?
- How do you define an interface?
- How do you use an interface?



What is an interface?

Why is an interface useful?

- ❑ An interface is a classlike construct that contains only **constants and abstract methods**.
- ❑ In many ways, an interface is similar to an abstract class, but the intent of an interface is to **specify common behavior for objects** of related classes or unrelated classes.



think



spokes
wheel
round

round
orange
fruit

ball
round
orange

spokes
wheel
round

Define an Interface

To distinguish an interface from a class, Java uses the following syntax to define an interface:

```
public interface InterfaceName {  
    constant declarations;  
    method signatures;  
}
```

Example:

```
public interface Edible {  
    /** Describe how to eat */  
    public abstract String howToEat();  
}
```



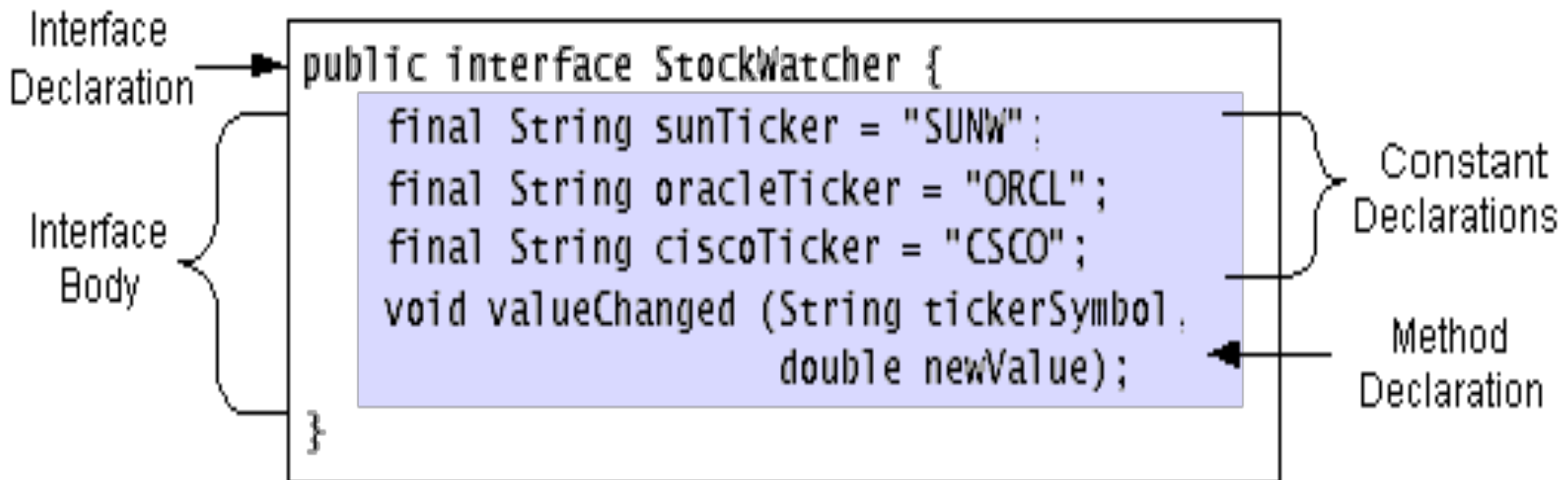
Interface is a Special Class

- An interface is treated like a special class in Java.
- **Like an abstract class, you cannot create an instance from an interface using the new operator**, but in most cases, you can use an interface more or less the same way you use an abstract class.
- For example, you can use an interface as a data type for a variable, as the result of casting, and so on.



Interface Body

- ❑ The interface body contains method declarations for **ALL** the methods included in the interface.
- ❑ A method declaration within an interface is followed by a semicolon (;)
- ❑ An interface does not provide implementations for the methods declared within it.



How to use interfaces

- You can write methods that work with more than one class

```
interface RuleSet {  
    boolean isLegal(Move m, Board b);  
    void makeMove(Move m);  
}
```

An interface cannot
implement any
methods

Every class that implements **RuleSet** must have these methods

- ```
class CheckersRules implements RuleSet { // one implementation
 public boolean isLegal(Move m, Board b) { ... }
 public void makeMove(Move m) { ... }
}
```
- ```
class ChessRules implements RuleSet { ... } // another implementation
```
- ```
class LinesOfActionRules implements RuleSet { ... } // and another
```

An interface is not part of the class hierarchy. Unrelated classes  
can implement the same interface.





# How to use interfaces

```
interface RuleSet {
 boolean isLegal(Move m, Board b);
 void makeMove(Move m);
}
```

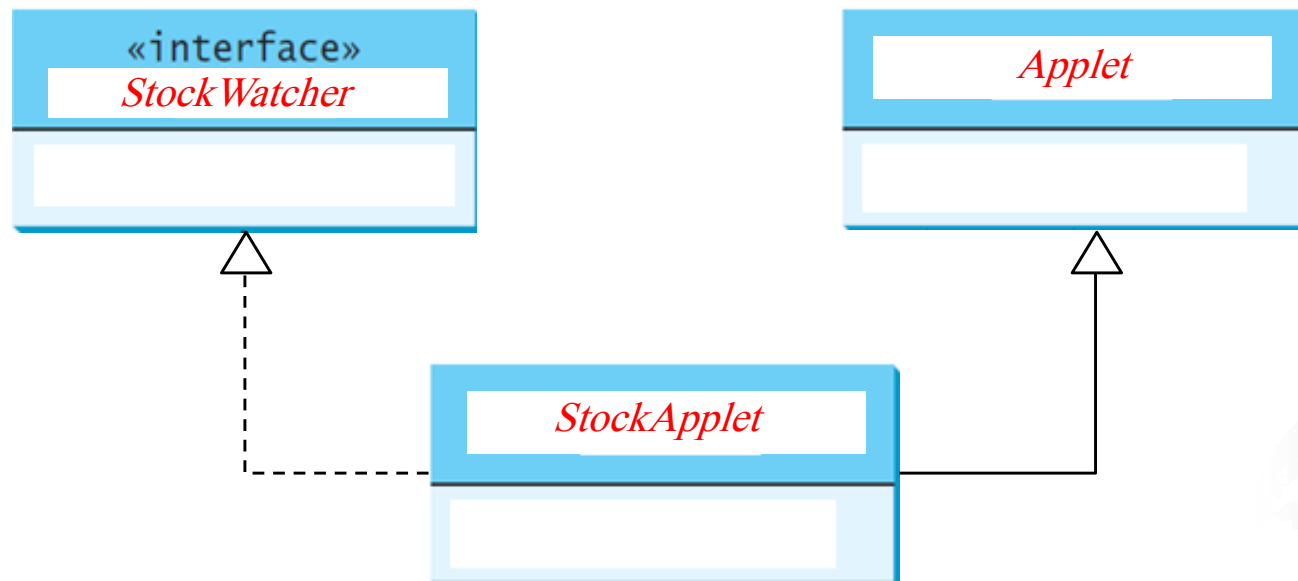
- ❑ `RuleSet rulesOfThisGame = new RuleSet();` ❌
  - This assignment is illegal because like an abstract class, you cannot create an instance from an interface using the new operator.
- ❑ `RuleSet rulesOfThisGame = new ChessRules();` ✅
  - This assignment is legal because you can use an interface as a data type for a variable, as the result of casting, and so on ( `rulesOfThisGame` object *is a* `RuleSet` object)
- ❑ `if (rulesOfThisGame.isLegal(m, b)) {rulesOfThisGame.makeMove(m); }` ✅
  - This statement is legal because, *whatever* kind of `RuleSet` object `rulesOfThisGame` is, it *must* have `isLegal` and `makeMove` methods.



# Example

public class StockApplet extends Applet **implements**  
StockWatcher {.....}

Complete



# Omitting Modifiers in Interfaces

All data fields are *public final static* and all methods are *public abstract* in an interface. For this reason, these modifiers can be omitted, as shown below:

```
public interface T1 {
 public static final int K = 1;

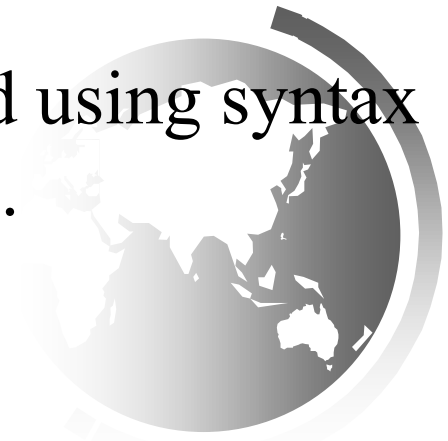
 public abstract void p();
}
```

Equivalent

```
public interface T1 {
 int K = 1;

 void p();
}
```

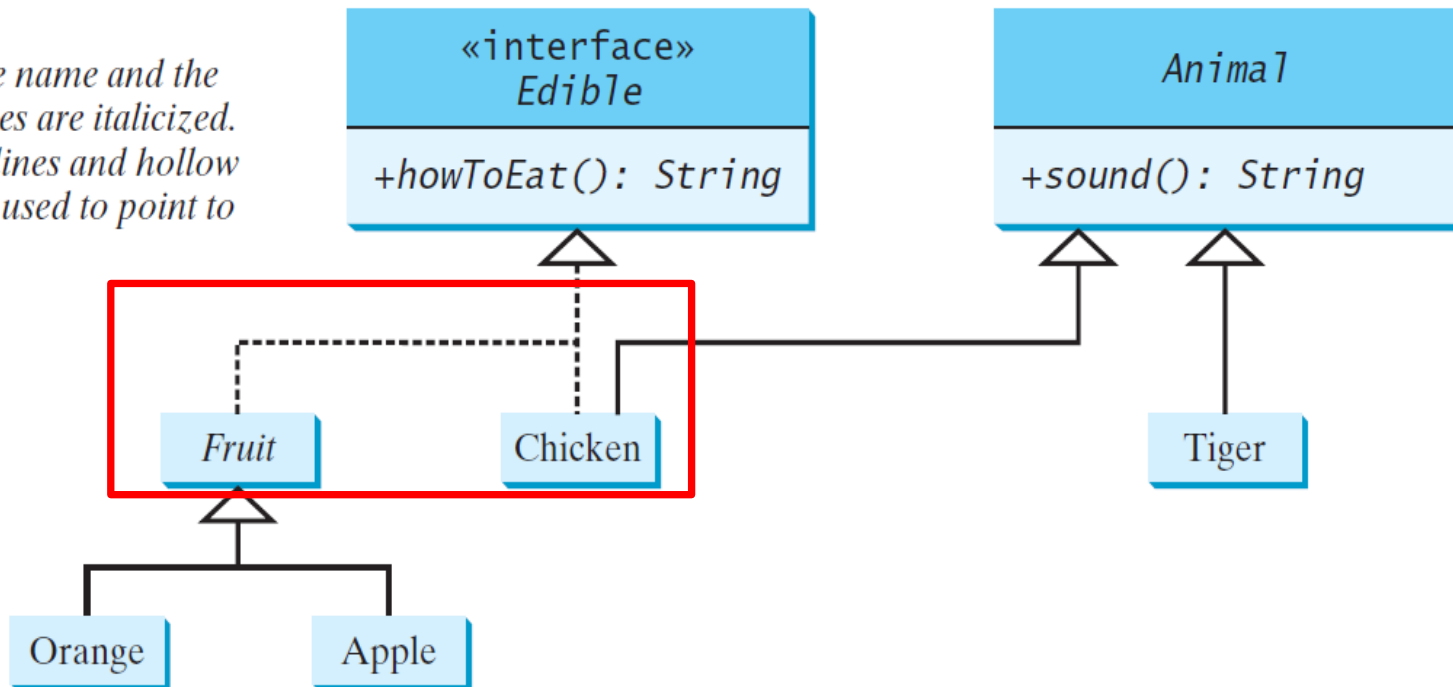
A constant defined in an interface can be accessed using syntax InterfaceName.CONSTANT\_NAME (e.g., T1.K).



# Example

*Notation:*

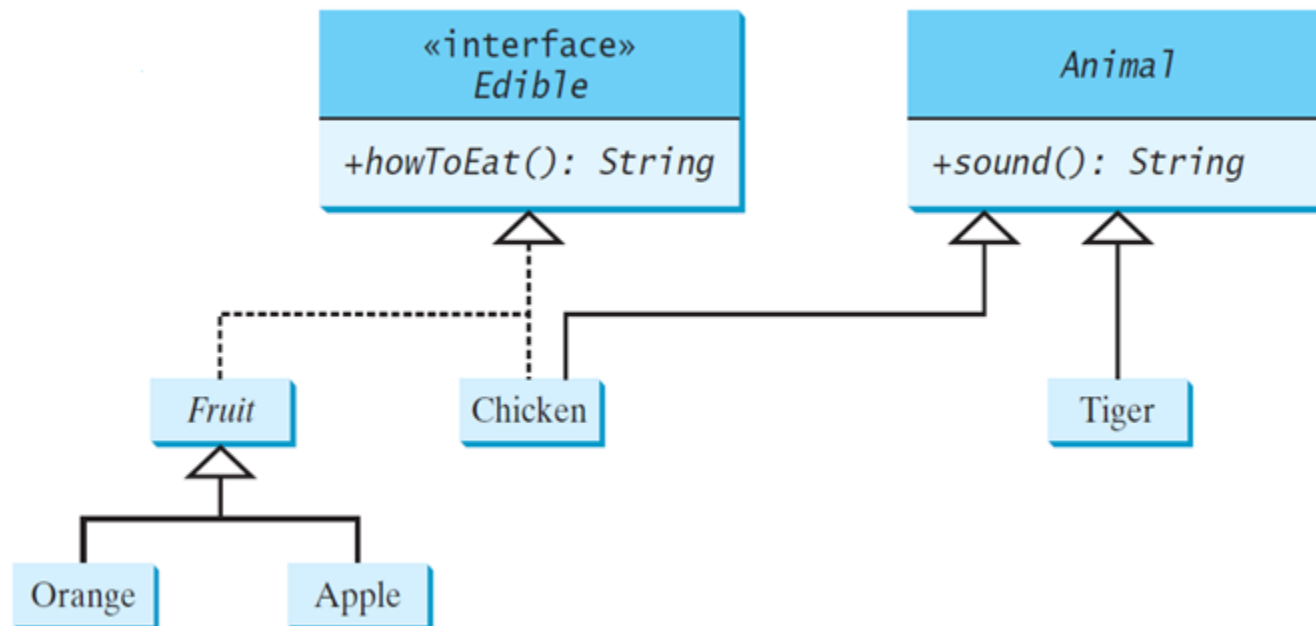
*The interface name and the method names are italicized.*  
*The dashed lines and hollow triangles are used to point to the interface.*





# Example

You can now use the Edible interface to specify whether an object is edible. This is accomplished by letting the class for the object implement this interface using the `implements` keyword. For example, the classes `Chicken` and `Fruit` implement the `Edible` interface (See `TestEdible`).



```
public interface Edible {
 /** Describe how to eat */
 public abstract String howToEat();
}
```

```
abstract class Animal {
 /** Return animal sound */
 public abstract String sound();
}
```

```
class Chicken extends Animal implements Edible {
 @Override
 public String howToEat() {
 return "Chicken: Fry it";
 }
}
```

```
@Override
public String sound() {
 return "Chicken: cock-a-doodle-doo";
}
}
```

```
abstract class Fruit implements Edible {
 // Data fields, constructors, and methods omitted here
}
```

```
class Tiger extends Animal {
 @Override
 public String sound() {
 return "Tiger: RROOAARR";
 }
}
```

```
class Apple extends Fruit {
 @Override
 public String howToEat() {
 return "Apple: Make apple cider";
 }
}
```

```
class Orange extends Fruit {
 @Override
 public String howToEat() {
 return "Orange: Make orange juice";
 }
}
```





```
public interface Edible {
 /** Describe how to eat */
 public abstract String howToEat();
}
```

```
abstract class Animal {
 /** Return animal sound */
 public abstract String sound();
}
```

```
class Chicken extends Animal implements Edible {
 @Override
 public String howToEat() {
 return "Chicken: Fry it";
 }
}
```

```
 @Override
 public String sound() {
 return "Chicken: cock-a-doodle-doo";
 }
}
```

```
abstract class Fruit implements Edible {
 // Data fields, constructors, and methods omitted here
}
```

```
public class TestEdible {
 public static void main(String[] args) {
 Object[] objects = {new Tiger(), new Chicken(), new Apple()};
 for (int i = 0; i < objects.length; i++) {
 if (objects[i] instanceof Edible)
 System.out.println(((Edible)objects[i]).howToEat());

 if (objects[i] instanceof Animal) {
 System.out.println(((Animal)objects[i]).sound());
 }
 }
 }
}
```

```
class Tiger extends Animal {
 @Override
 public String sound() {
 return "Tiger: RROOAARR";
 }
}
```

```
class Apple extends Fruit {
 @Override
 public String howToEat() {
 return "Apple: Make apple cider";
 }
}
```

```
class Orange extends Fruit {
 @Override
 public String howToEat() {
 return "Orange: Make orange juice";
 }
}
```

```
public class TestEdible {
 public static void main(String[] args) {
 Object[] objects = {new Tiger(), new Chicken(), new Apple()};
 for (int i = 0; i < objects.length; i++) {
 if (objects[i] instanceof Edible)
 System.out.println(((Edible)objects[i]).howToEat());

 if (objects[i] instanceof Animal) {
 System.out.println(((Animal)objects[i]).sound());
 }
 }
 }
}
```

Output:

```
Tiger: RROOAARR
Chicken: Fry it
Chicken: cock-a-doodle-doo
Apple: Make apple cider
```



assume LittleChicken is a subtype of Chicken. For each question, answer if the code can compile, can run. If not, give a reason. If it runs, give the output.

a. `Edible x = new Tiger();`

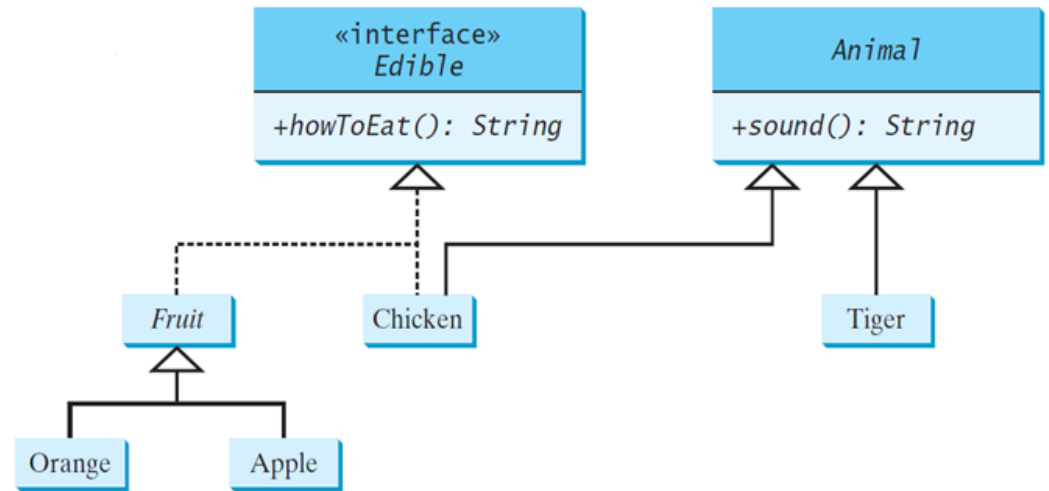
b. `Edible x = new Chicken();`  
`System.out.println(x.sound());`

c. `Edible x = new Chicken();`  
`System.out.println((Animal)x.sound());`

d. `Edible x = new Chicken();`  
`System.out.println(((Animal)x).sound());`

e. `Edible x = new LittleChicken();`  
`System.out.println(x.howToEat());`

f. `LittleChicken x = new Chicken();`



assume LittleChicken is a subtype of Chicken. For each question, answer if the code can compile, can run. If not, give a reason. If it runs, give the output.

a. Edible x = new Tiger();

b. Edible x = new Chicken();  
System.out.println(x.sound());

c. Edible x = new Chicken();  
System.out.println((Animal)x.sound());

d. Edible x = new Chicken();  
System.out.println(((Animal)x).sound());

e. Edible x = new LittleChicken();  
System.out.println(x.howToEat());

f. LittleChicken x = new Chicken();

a. Compile error. Tiger is not Edible.

b. Compile error. x is declared Edible, but Edible does not have the sound() method.

c. Compile error. x.sound() performed first before casting. x does not have the sound() method.

d. Chicken: cock-a-doodle-doo

e. Chicken: Fry it

f. Compile error. Chicken is not a LittleChicken.

Cannot assign a Chicken object to a variable of type LittleChicken.



# Question

Suppose A is an interface. Can you create an instance using `new A()`?



# Answer

No



# Question

Suppose *A* is an interface. Can you declare a reference variable *x* with type *A* like this?

```
A x;
```



# Answer

Yes





Which of the following is a correct interface?

(a)  
interface A {  
 void print() { }  
}

(b)  
abstract interface A {  
 abstract void print() { }  
}

(c)  
abstract interface A {  
 print();  
}

(d)  
interface A {  
 void print();  
}

(e)  
interface A {  
 default void print() {  
 }  
}

(f)  
interface A {  
 static int get() {  
 return 0;  
 }  
}



Which of the following is a correct interface?

(a)  
interface A {  
 void print() { }  
}

(b)  
abstract interface A {  
 abstract void print() { }  
}

(c)  
abstract interface A {  
 print();  
}

(d)  
interface A {  
 void print();  
}

(e)  
interface A {  
 default void print() {  
 }  
}

(f)  
interface A {  
 static int get() {  
 return 0;  
 }  
}

(a) is wrong, because the print() method has a body.

(b) is wrong, because the interface cannot have the abstract keyword.

(c) is wrong, because the interface cannot have the abstract keyword.

(d) is correct.

(e) is correct. JDK 8 allows default methods in the interface.

(f) is correct. JDK 8 allows static methods in the interface.



# Question

Show the error in the following code:

```
interface A {
 void m1();
}
```

```
class B implements A {
 void m1() {
 System.out.println("m1");
 }
}
```



# Answer

All methods defined in an interface are public. When a class implements the interface, the method must be declared public. The visibility cannot be reduced.



# The Comparable Interface

```
// This interface is defined in
// java.lang package
package java.lang;
```

```
public interface Comparable<E> {
 public int compareTo(E o);
}
```

*The **Comparable** interface defines the **compareTo** method for comparing objects*



# The toString and equals Methods

- ❑ Each wrapper class overrides the `toString` and `equals` methods defined in the `Object` class.
- ❑ Since all the numeric wrapper classes and the `Character` class **implement the `Comparable` interface**, the `compareTo` method is implemented in these classes.
- ❑ The **`compareTo`** method determines the order of this object with the specified object `o` and returns a negative integer, zero, or a positive integer if this object is less than, equal to, or greater than `o`.
- ❑ The **`Comparable`** interface is a generic interface.

# Integer and BigInteger Classes

```
public class Integer extends Number
 implements Comparable<Integer> {
 // class body omitted

 @Override
 public int compareTo(Integer o) {
 // Implementation omitted
 }
}
```

```
public class BigInteger extends Number
 implements Comparable<BigInteger> {
 // class body omitted

 @Override
 public int compareTo(BigInteger o) {
 // Implementation omitted
 }
}
```

# String and Date Classes

```
public class String extends Object
 implements Comparable<String> {
 // class body omitted

 @Override
 public int compareTo(String o) {
 // Implementation omitted
 }
}
```

```
public class Date extends Object
 implements Comparable<Date> {
 // class body omitted

 @Override
 public int compareTo(Date o) {
 // Implementation omitted
 }
}
```

# Question

□ What is the output?

- 1 `System.out.println(new Integer(3).compareTo(new Integer(5)));`
- 2 `System.out.println("ABC".compareTo("ABC"));`
- 3 `java.util.Date date1 = new java.util.Date(2013, 1, 1);`
- 4 `java.util.Date date2 = new java.util.Date(2012, 1, 1);`
- 5 `System.out.println(date1.compareTo(date2));`





# Answer

-1 since 3 less than 5

0 since ABC equal ABC

1 since date1 is greater than date2



# instanceof

- **instanceof** is a keyword that tells you whether a variable “is a” member of a **class** or an **interface**.

- For example, if

```
class Dog extends Animal implements Pet {...}
Animal fido = new Dog();
```

then the following are true or false:

fido instanceof Dog ✓

fido instanceof Animal ✓

fido instanceof Pet ✓



# instanceof

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object, then all the following expressions are true.

|   |                   |            |
|---|-------------------|------------|
| n | <b>instanceof</b> | Integer    |
| n | <b>instanceof</b> | Object     |
| n | <b>instanceof</b> | Comparable |

|   |                   |            |
|---|-------------------|------------|
| s | <b>instanceof</b> | String     |
| s | <b>instanceof</b> | Object     |
| s | <b>instanceof</b> | Comparable |

|   |                   |                |
|---|-------------------|----------------|
| d | <b>instanceof</b> | java.util.Date |
| d | <b>instanceof</b> | Object         |
| d | <b>instanceof</b> | Comparable     |



# Generic sort Method

The `java.util.Arrays.sort(array)` method requires that the elements in an array are instances of `Comparable<E>`.



SortComparableObjects

Run

Since all `Comparable` objects have the `compareTo` method, the `java.util.Arrays.sort(Object[])` method in the Java API uses the `compareTo` method to compare and sorts the objects in an array, provided that the objects are instances of the `Comparable` interface



# Generic sort Method

```
1 import java.math.*;
2
3 public class SortComparableObjects {
4 public static void main(String[] args) {
5 String[] cities = {"Savannah", "Boston", "Atlanta", "Tampa"};
6 java.util.Arrays.sort(cities);
7 for (String city: cities)
8 System.out.print(city + " ");
9 System.out.println();
10
11 BigInteger[] hugeNumbers = {new BigInteger("2323231092923992"),
12 new BigInteger("432232323239292"),
13 new BigInteger("54623239292")};
14 java.util.Arrays.sort(hugeNumbers);
15 for (BigInteger number: hugeNumbers)
16 System.out.print(number + " ");
17 }
18 }
```

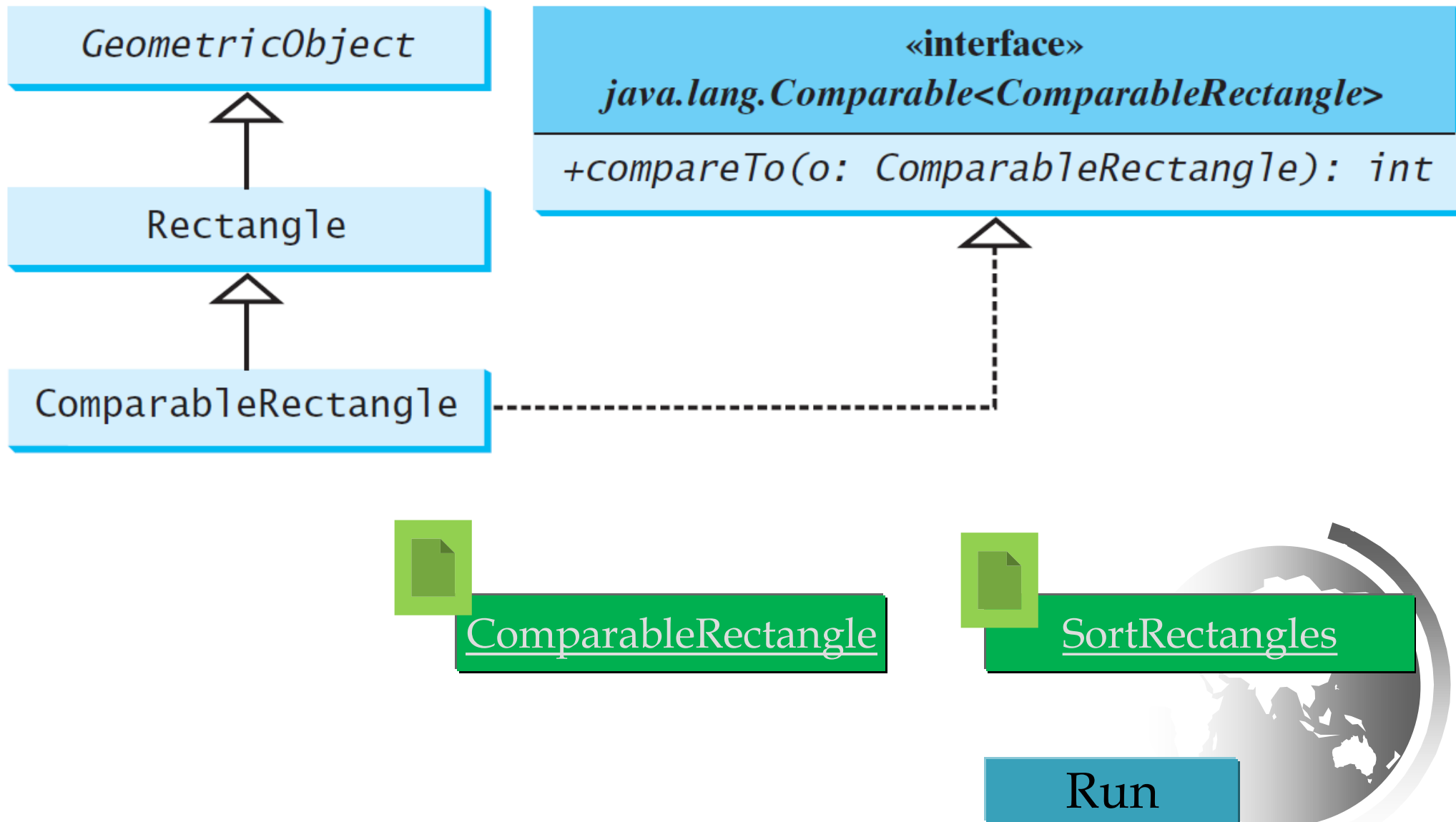
create an array  
sort the array

create an array

sort the array

Atlanta Boston Savannah Tampa  
54623239292 432232323239292 2323231092923992

# Defining Classes to Implement Comparable



```

public class ComparableRectangle extends Rectangle
 implements Comparable<ComparableRectangle> {
 /** Construct a ComparableRectangle with specified properties */
 public ComparableRectangle(double width, double height) {
 super(width, height);
 }

 @Override // Implement the compareTo method defined in Comparable
 public int compareTo(ComparableRectangle o) {
 if (getArea() > o.getArea())
 return 1;
 else if (getArea() < o.getArea())
 return -1;
 else
 return 0;
 }

 @Override // Implement the toString method in GeometricObject
 public String toString() {
 return "Width: " + getWidth() + " Height: " + getHeight() +
 " Area: " + getArea();
 }
}

```

```
public class SortRectangles {
 public static void main(String[] args) {
 ComparableRectangle[] rectangles = {
 new ComparableRectangle(3.4, 5.4),
 new ComparableRectangle(13.24, 55.4),
 new ComparableRectangle(7.4, 35.4),
 new ComparableRectangle(1.4, 25.4)};
 java.util.Arrays.sort(rectangles);
 for (Rectangle rectangle: rectangles) {
 System.out.print(rectangle + " ");
 System.out.println();
 }
 }
}
```

Width: 3.4 Height: 5.4 Area: 18.36

Width: 1.4 Height: 25.4 Area: 35.5599999999999995

Width: 7.4 Height: 35.4 Area: 261.96

Width: 13.24 Height: 55.4 Area: 733.496



# Question

What is wrong in the following code?

```
public class Test {
 public static void main(String[] args) {
 Person[] persons = {new Person(3), new Person(4), new
Person(1)};
 java.util.Arrays.sort(persons);
 }
}

class Person {
 private int id;

 Person(int id) {
 this.id = id;
 }
}
```



# Answer

What is wrong in the following code?

```
public class Test {
 public static void main(String[] args) {
 Person[] persons = {new Person(3), new Person(4), new
Person(1)};
 java.util.Arrays.sort(persons);
 }
}

class Person {
 private int id;

 Person(int id) {
 this.id = id;
 }
}
```

The Person class does not implement the Comparable interface, two persons cannot be compared using the compareTo method.



# Question

Can the following code be compiled?  
Why?

```
Integer n1 = new Integer(3);
Object n2 = new Integer(4);
System.out.println(n1.compareTo(n2));
```



# Answer

Can the following code be compiled? Why?

```
Integer n1 = new Integer(3);
Object n2 = new Integer(4);
System.out.println(n1.compareTo(n2));
```

**n1 is an Integer object whose compareTo method require an Integer argument, but n2 is declared as Object. The compiler will raise an error.**



# Recap

What is an abstract class?

What is an interface?



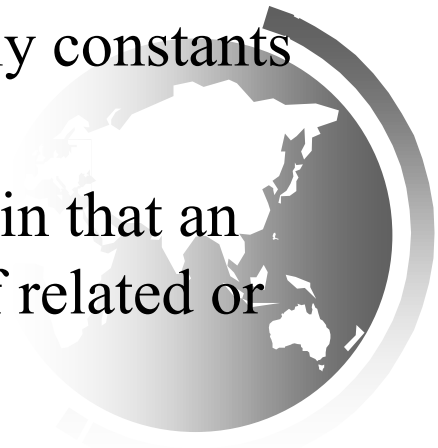
# Recap

## What is an abstract class?

- is a **restricted class that cannot be used to create objects**. To access it, it must be inherited from another class.
- contains regular methods and abstract methods.
- Abstract method: can only be used in an abstract class, and it does not have a body. The body is provided by the subclass.

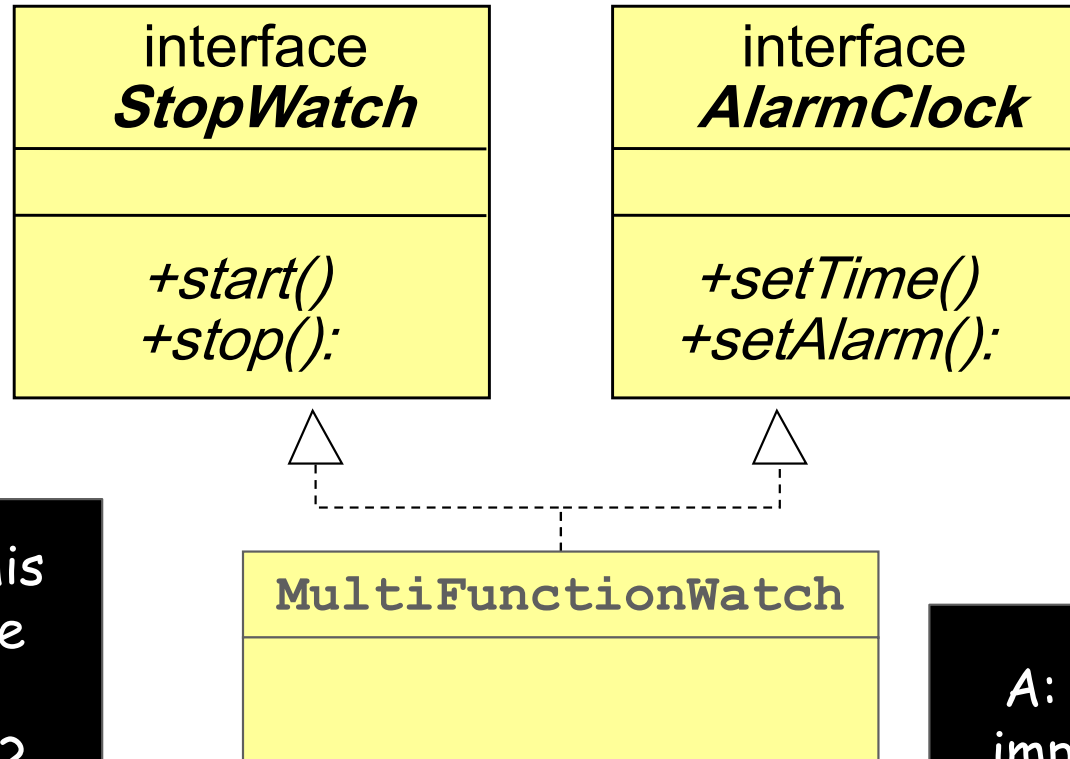
## What is an interface?

- An interface is a class-like construct that contains only constants and abstract methods.
- An interface is similar to an abstract class but differs in that an interface can specify common behavior for objects of related or unrelated classes.



# Multiple Interfaces

- Classes are allowed to implement multiple interfaces.

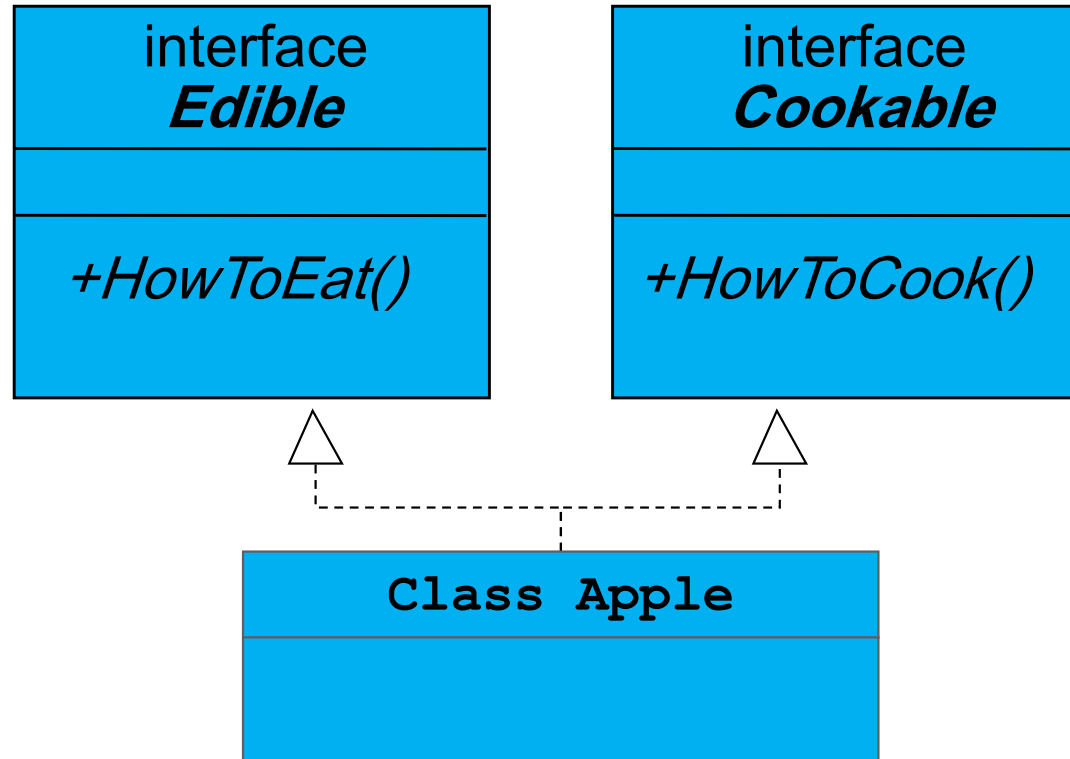


Q: Why is this not the same as multiple inheritance?

A: There is no implementation to inherit

# Multiple Interfaces

- Classes are allowed to implement multiple interfaces.





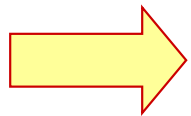
# Conflict Resolution Rules

A conflict can occur when a class implements multiple interfaces with either same constant variable or methods with same names.



# Conflict Resolution Rules

- Classes can implement multiple interfaces.



**Conflict information** : two same constants with different values or two methods with same signature but different return type.

## □ Conflict Resolution:

- Same constants with different values: **compile error**
- Same signature and different return type: **compile error**
- Same signature and return type: **same method**
- different signatures: **overloading**



# Conflict Resolution Rules

## 1) The interfaces has the same constant variable

- Same value

- Example: `interface a {int x=10;}`  
`interface b {int x=10;}`

- `class test implements a,b {}` ❌

- Different value

The reference to x is ambiguous!

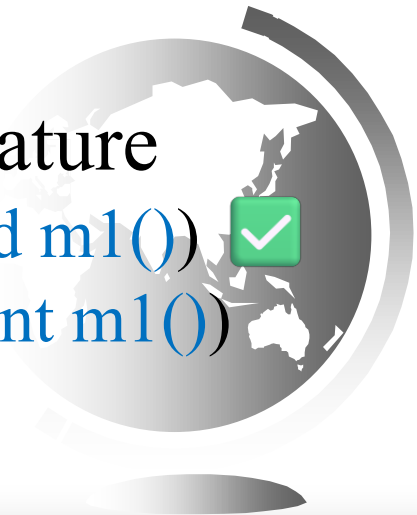
- Example: `interface a {int x=10;}`  
`interface b {int x=1000;}`

- `class test implements a,b {}` ❌

## 2) The interfaces has methods with same signature

- Same return type (Example, `void m1()` `void m1()`) ✓

- Different return type (Example, `void m1()` `int m1()`)



# Conflict Resolution Rules

1) The interfaces has the same constant variable:

– Same value

```
interface a {int x=10;}
```

```
interface b {int x=10;}
```

```
public class test implements a,b {
```

```
 public static void main(String[] args) {
```

```
 System.out.println(a.x);
```



10 because we specified an interface

```
 System.out.println(b.x);
```



10 because we specified an interface

```
 System.out.println(x);
```

✗ error: reference to x is ambiguous,  
both variable x in a and variable x in b match

```
 }
```



# Conflict Resolution Rules

1) The interfaces has the same constant variable:

– Different values

```
interface a {int x=10;}
```

```
interface b {int x=100;}
```

```
public class test implements a,b {
```

```
 public static void main(String[] args) {
```

```
 System.out.println(a.x);
```



10

```
 System.out.println(b.x);
```



100

```
 System.out.println(x);
```

✗ error: reference to x is ambiguous,  
both variable x in a and variable x in b match

```
 }
```



# Abstract Classes vs. Interfaces

## Abstract Classes

- ❑ Can have data fields.
- ❑ Methods may have an implementation.
- ❑ Classes and abstract classes extend abstract classes.
- ❑ A class cannot extend multiple abstract classes.

## Interfaces

- ❑ Can only have constants.
- ❑ Methods have **no** implementation.
- ❑ Classes and abstract classes implement interfaces.
- ❑ Interfaces can extend multiple interfaces.
- ❑ A class can implement multiple interfaces.

### *Variables*

Abstract class    No restrictions.

Interface    All variables must be  
**public static final**.

### *Constructors*

Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator.

No constructors. An interface cannot be instantiated using the new operator.

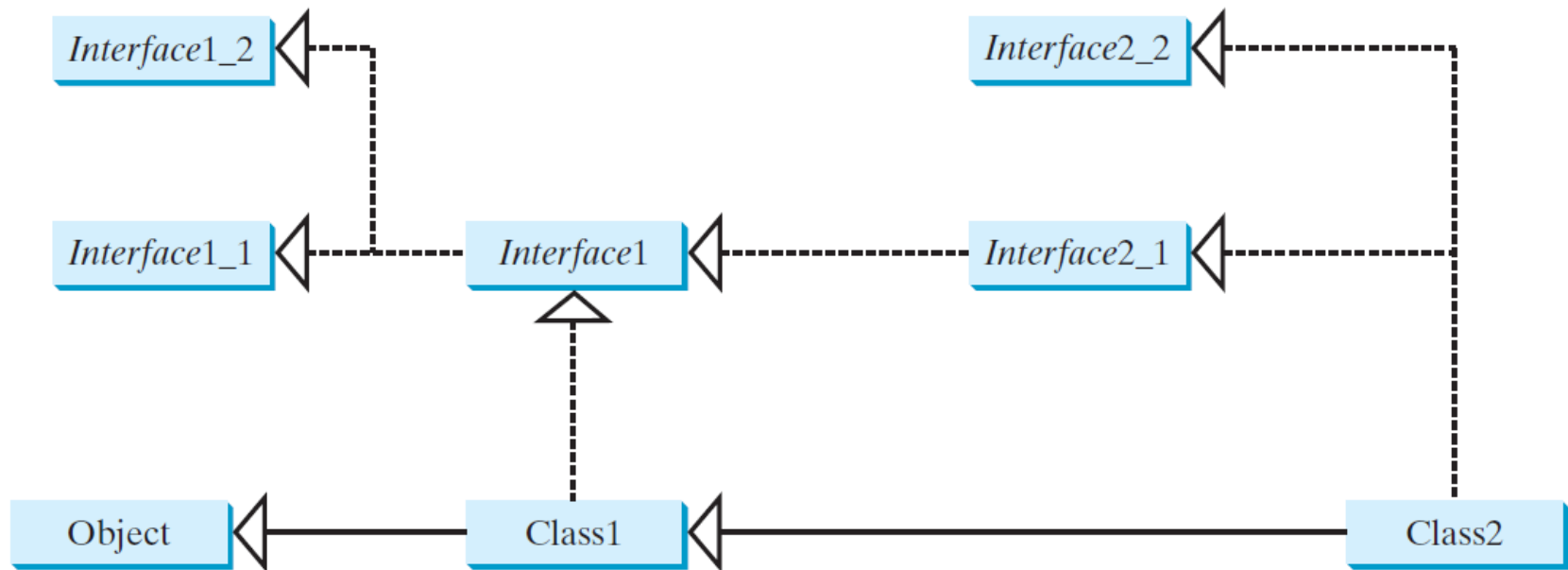
### *Methods*

No restrictions.

All methods must be public abstract instance methods

# Interfaces vs. Abstract Classes, cont.

All classes share a single root, the Object class, but there is no single root for interfaces.



Suppose that *c* is an instance of **Class2**. *c* is also an instance of **Object**, **Class1**, **Interface1**, **Interface1\_1**, **Interface1\_2**, **Interface2\_1**, and **Interface2\_2**.

# Whether to use an interface or a class?

Abstract classes and interfaces can both be used to model common features.

How do you decide whether to use an interface or a class?

- In general, a **strong is-a relationship** that clearly describes a **parent-child relationship** should be modeled using **classes**.
- A **weak is-a relationship** can be modeled using **interfaces**. A weak is-a relationship, also known as an **is-kind-of relationship**, indicates that an object possesses a certain property.
- You can also use interfaces to avoid single inheritance restriction if multiple inheritance is desired.
- In the case of multiple inheritance, you have to design one as a superclass, and others as interfaces.





# Abstract Classes or Interfaces?

How do you decide whether to use abstract classes or interfaces?

- If there is common implementation -> abstract class
- If there is no common implementation -> interface
  - Interfaces allow classes implementing multiple interfaces...





How do you decide whether to use an interface or a class?

- A staff member is a person => interface or class??
- All strings are comparable=> interface or class??

The String class implements the Comparable interface.



# Class Design Guidelines:

## (Coherence)

- A class should describe a single entity, and all the class operations should logically fit together to support a coherent purpose.
- You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.



# Class Design Guidelines:

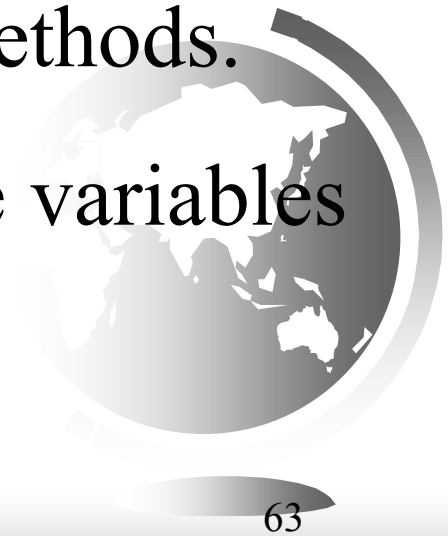
## (Separating responsibilities)

- A single entity with too many responsibilities can be broken into several classes to separate responsibilities.
- The classes `String`, `StringBuilder`, and `StringBuffer` all deal with strings, for example, but have different responsibilities.
  - The `String` class deals with immutable strings, the `StringBuilder` class is for creating mutable strings, and the `StringBuffer` class is similar to `StringBuilder` except that `StringBuffer` contains synchronized methods for updating strings.

# Class Design Guidelines:

## (Consistency)

- Follow standard Java programming style and naming conventions.
- Choose informative names for classes, data fields, and methods.
- Always place the data declaration before the constructor, and place constructors before methods.
- Always provide a constructor and initialize variables to avoid programming errors.



# Class Design Guidelines:

## (Consistency)

- Provide a public no-arg constructor. If no constructors are defined explicitly, a public default no-arg constructor with an empty body is assumed.
- Override the equals method and the toString method defined in the Object class whenever possible.



# Class Design Guidelines:

## (Clarity)

- Classes are designed for reuse. Users can incorporate classes in many different combinations, orders, and environments.
- Therefore, you should design a class that imposes no restrictions on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.



# Class Design Guidelines:

## (Encapsulation)

- A class should use the private modifier to hide its data from direct access by clients.
- You can use get methods and set methods to provide users with access to the private data, but only to private data you want the user to see or to modify.
- A class should also hide methods not intended for client use.

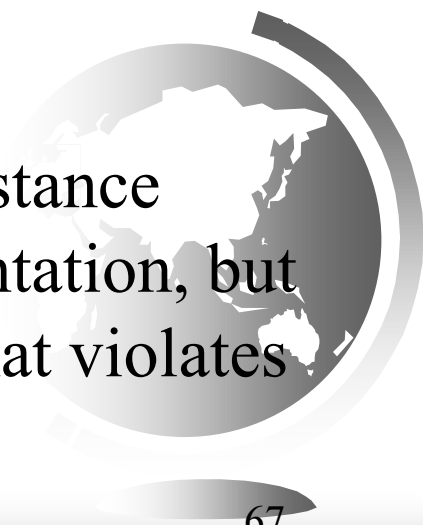




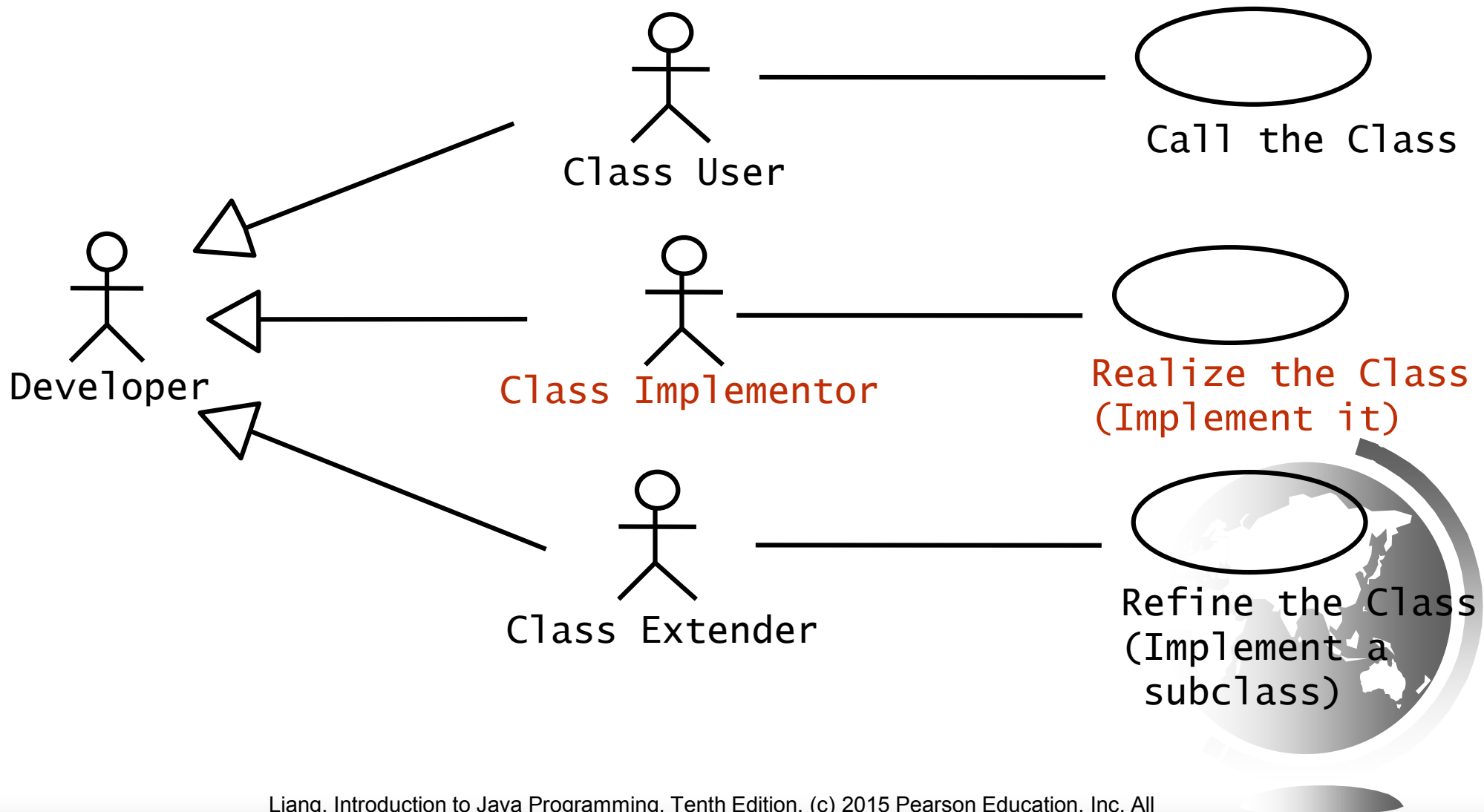
# Class Design Guidelines:

## (Encapsulation)

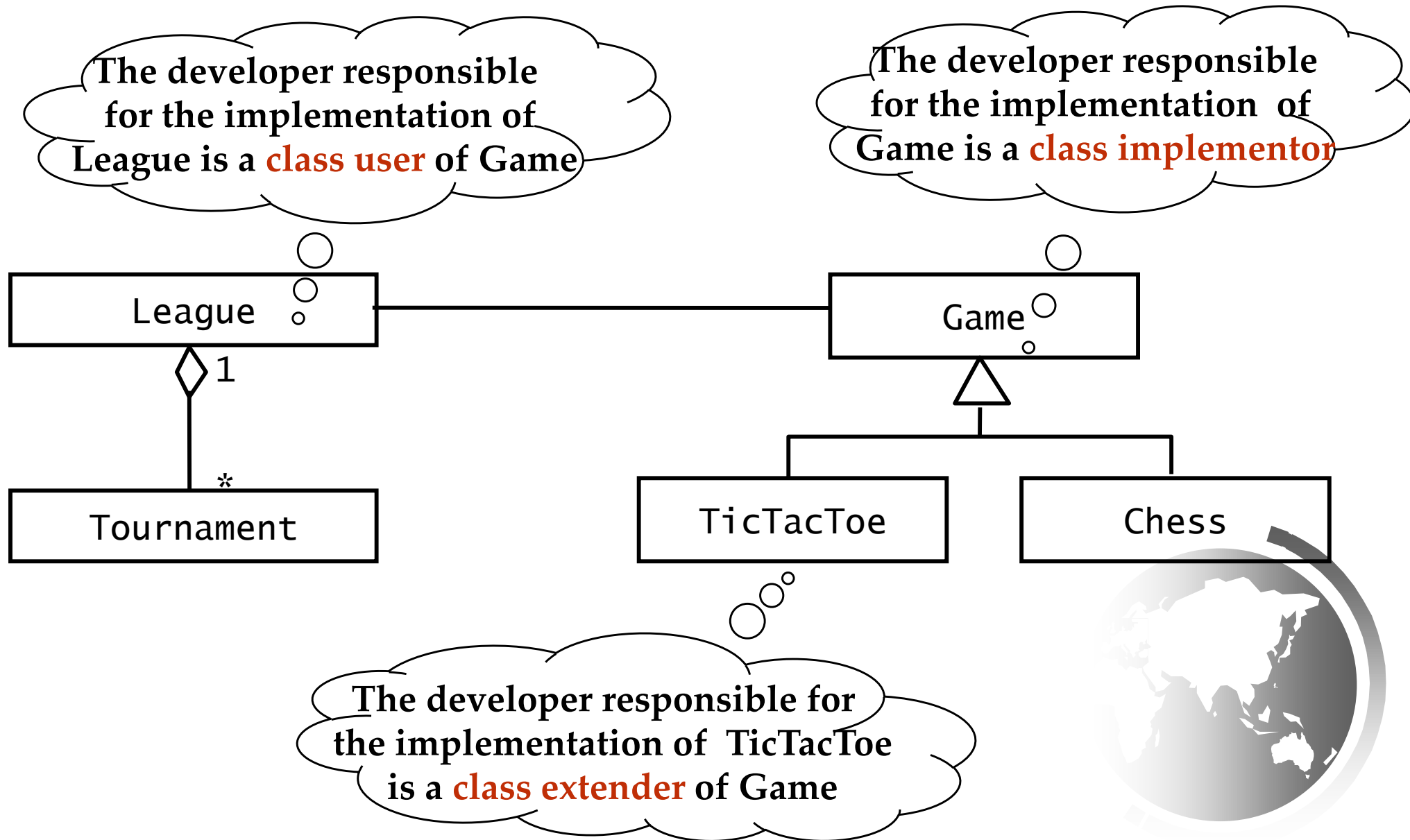
- Each class can present two contracts – one for the users of the class and one for the extenders of the class.
  - Make the fields private and accessor methods public if they are intended for the users of the class.
  - Make the fields or method protected if they are intended for extenders of the class.
- The contract for the extenders encompasses the contract for the users.
- The extended class may increase the visibility of an instance method from protected to public, or change its implementation, but you should never change the implementation in a way that violates that contract.



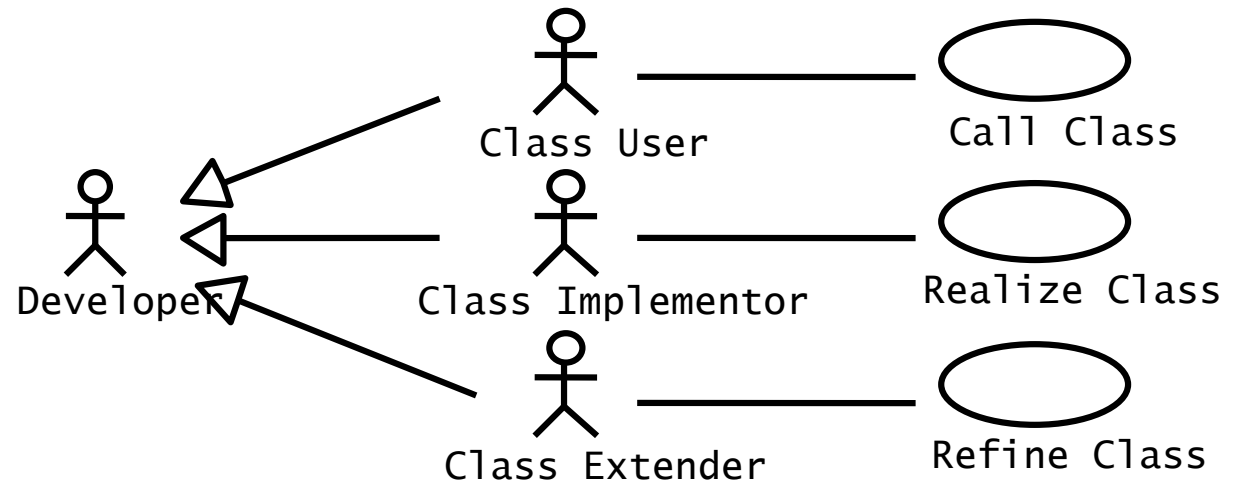
# Developers play 3 different Roles during the Design of a Class



# Class User versus Class Extender



# Add Visibility Information



## Class user (“Public”): +

- Public attributes/operation can be accessed by any class.

## Class implementor (“Private”): -

- Private attributes and operations can be accessed only by the class in which they are defined.
- They cannot be accessed by subclasses or other classes.

## Class extender (“Protected”): #

- Protected attributes/operations can be accessed by the class in which they are defined and by any descendent of the class.



# Class Design Guidelines:

## (Inheritance vs. Aggregation)

- The difference between **inheritance** and **aggregation** is the difference between an **is-a** and a **has-a** relationship.
- For example, an *apple is a fruit*; thus, you would use inheritance to model the relationship between the classes **Apple** and **Fruit**.
- A *person has a name*; thus, you would use aggregation to model the relationship between the classes **Person** and **Name**.



# Class Design Guidelines:

## (Instance vs. Static)

- A variable or method that is dependent on a specific instance of the class must be an instance variable or method.
- A variable that is shared by all the instances of a class should be declared static.
- Always reference static variables and methods from a class name (rather than a reference variable) to improve readability and avoid errors.



1

```
public interface T1 {
 public static final int K = 1;

 public abstract void p();
}
```

Equivalent

```
public interface T1 {
 int K = 1;

 void p();
}
```

2

3

All data fields are *public static final* and all methods are *public abstract* in an interface.

- 4 All methods defined in an interface are public. When a class implements the interface, the method must be declared public. The visibility cannot be reduced.
- 5 An interface cannot implement any methods except static methods.
- 6 Every class that implements **interface** must implement all interface methods.
- 7 A constant defined in an interface can be accessed using syntax InterfaceName.CONSTANT\_NAME (e.g., T1.K).